Cross-Assembler für C8205 / C8205Z Version 0.03-001

Die gesamte Software läuft sowohl unter Linux als auch Windows 10.
Die Software ist in C (gcc) programmiert, die grafische Oberfläche benutzt node.js

1. Installation

Zum Zweck der Installation ist die Datei C8205.zip in einem geeigneten Verzeichnis zu entpacken. Beim Entpacken wird ein Unterverzeichnis angelegt, in dem sich sowohl Windows- als auch Linux-Binaries befinden.

1.1 Verzeichnisstruktur

Der Assembler liegt im Unterverzeichnis Assembler des Gesamtprojekts C8205. Es enthält folgende Struktur (Beispielprojekte gekürzt):

```
make_c82ass.sh          # Skript zum Erzeugen des Assemblers unter Linux
c82ass.sh               # Ausführen des Assemblers
make_c82ass.bat         # Batch-Datei zum Erzeugen des Assemblers unter Windows
                          (Pendant zu make_c82ass.sh)
Dokumentation           # diese Dokumentation
_Linux                  # Linux-Binary des Assemblers
projekdetails           # zuletzt benutzte Aufrufparameter
projekdetails.<Projekt> # gesicherte Aufrufparameter je Projekt
Projekt_1               # Projektverzeichnis (Beispiel, mit Unterverzeichnissen)
Proj_FeldSortieren      # Projektverzeichnis (Beispiel, ohne Unterverzeichnisse)
Sourcecode              # C-Quellcode des Assemblers selbst
_Windows                # Windows-Binary des Assemblers
```

./Assembler/Dokumentation:
C8205_Assembler_Dokumentation.md C8205_Assembler_Dokumentation.odt C8205_Assembler_Dokumentation.pdf

./Assembler/_Linux:
c82ass

./Assembler/Projekt_1:
bsp_5.15
bsp_5.16

./Assembler/Projekt_1/bsp_5.16:
bsp_5_16.asm bsp_5_16.dmp bsp_5_16.log bsp_5_16.mqo bsp_5_16.obj bsp_5_16.sym bsp_5_16.test

./Assembler/Proj_FeldSortieren:
code.asm code_wdhl_bef.asm code.log code.mqo code.obj code.sym code.test

./Assembler/Sourcecode:
c8205_op_codes.c c82_austausch.c c82_austausch.h ca82ass.c ca82_protokoll.c ca82_protokoll.h ca82_standards.c ca82_standards.h c82ass.makros

./Assembler/_Windows:

Projektverzeichnisse können beliebig benannt sein, auch Unterverzeichnisse enthalten. In Ihnen steht der Quelltext , die Erzeugte Objektdatei (meint die K1K2- ladefähige Textdatei) und weitere Hilfsdateien. Die Programme c82ass, c82load, c82dump, c82simi erwarten ihre Dateien immer in einem Projektunterverzeichnis, das im gleichen Pfad wie sie selbst liegen muß.
In der vorliegenden Projektstruktur ist nur c82ass enthalten. Die Programme c82load, c82dump, c82simi und der C8205Simulator sind nicht Teil dieses Projekts und liegen nicht bei; sie werden in dieser Dokumentation nur erwähnt, weil c82ass Dateien für sie erzeugt (.mqo und .obj für c82load, .mqo für c82simi, .sym für c82dump). Für die Prüfung von Assemblerergebnissen im Interpreter dient stattdessen die Datei .test (s. C8205-Interpreter-Durchführung_eines_Tests.md, Abschnitt 2a).

make_c82ass.sh ist ein Shell-Script zum Erzeugen des Assemblers . Die Datei make_c82ass.bat leistet das gleiche für den MinGW-w64 Crosscompiler unter Windows.
Der Pfad _Linux enthält alle Linux-Binaries und Libraries, der Pfad _Windows die Pendants für Windows 10.
Sourcecode enthält ausschließlich den C-Quellcode.

2. **Überblick**

Sowohl das Programm c82ass.sh (Linux) als auch das Windows-Binary _Windows\c82ass.exe (die Datei make_c82ass.bat erzeugt es nur) werden über Aufrufparameter versorgt. Bei jedem Start eines der Programme werden die jeweils letztbenutzten Parameter automatisch in der Datei projekdetails abgelegt, so dass ein Folgestart ohne Angabe von Parametern gelingt. Zur Änderung müssen die Parameter einfach mit veränderten Werten wieder angegeben werden.

$c82ass.sh -h
C8205 Assembler C82aas Version 0.2
Usage: ./_Linux/c82ass -p Projektverzeichnis -s Quellcode-Datei

Im Usage-Text steht der Aufrufname (argv[0]); über c82ass.sh erscheint daher ./_Linux/c82ass. c82ass.sh ruft ./_Linux/c82ass relativ auf und muss deshalb aus dem Verzeichnis Assembler gestartet werden; dort werden auch projekdetails und (zuerst) c82ass.makros gesucht.

-p nennt das Verzeichnis, in dem die .asm-Datei direkt liegt (z. B. Projekt_1/bsp_5.16). Unterverzeichnisse werden nicht durchsucht; ein Pfadanteil bei -s wird abgeschnitten.
Hat der Name bei -s keine Endung aus genau drei Zeichen, wird .asm angehängt (bsp_5_16 wird zu bsp_5_16.asm).
projekdetails enthält die Zeilen -p <Verzeichnis> und -s <Datei>; fehlt einer der Parameter im Aufruf, wird der gespeicherte Wert benutzt. Beim Wechsel des Projektverzeichnisses wird die bisherige Datei als projekdetails.<altes Projekt> gesichert (Schrägstriche im Namen werden zu _, z. B. projekdetails.Projekt_1_bsp_5.16) und die gesicherte Datei des neuen Projekts, falls vorhanden, zurückgeholt.
Rückgabecode: 0 = fehlerfrei, 16 = Fehler (jede Art von Abbruch). Parameter ab einem Argument, das mit > beginnt, werden ignoriert.

3. Der Assembler c82ass

Version: 0.03

1. Programmstart

Der Assembler **c82ass** erwartet eine Eingabedatei mit <quelltext> unter einem Dateinamen **.asm**.
Er liest im Pass 0 die Makrodatei **c82ass.makros** (fester Name; gesucht zuerst im Arbeitsverzeichnis des Aufrufs, sonst in Sourcecode/), sammelt die im Quelltext definierten Makros (siehe Abschnitt 4.1), prüft die Makrodefinitionen und löst die Makroaufrufe im Quelltext auf (siehe Abschnitt 4).
Er ermittelt im Pass 1 die Literaltabelle, die im Logfile **.log** ausgegeben wird.
Er ermittelt im Pass 2 die Symboltabelle, die ebenfalls im Logfile **.log** ausgegeben wird, und schreibt die Symboldatei **.sym**.
Er erzeugt im Pass 3 aus dem aufgelösten Quelltext (Literale und Symbole eingesetzt) die Kennzeichenbefehle für das „Kleine Eingabeprogramm“, die im Objektfile **.obj** ausgegeben werden. Im selben Pass entstehen die Hilfsdatei **.mqo**, die formatierte Speicherbelegung **.dmp** und die Trommeldatei **.test** für den Interpreter.
Auf STDOUT gibt der Assembler dabei den formatierten Quelltext (einschließlich der aufgelösten Literale) aus.

Der Aufruf erfolgt zum Beispiel in der Form:

$c82ass.sh -p PzBeispiele_4 -s bsp_5.17_Oktaldruck.asm

Fehler aus den Pässen stehen in der Ausgabe des formatierten Quelltextes auf STDOUT als Zeile „;*** (<Pass>) Fehler: …“; Meldungen mit „Fehler:“, „Warnung:“ und „Abbruch“ sowie die Statuszeilen „Projektverzeichnis:“, „Benutze für -s die Datei:“ und „Startadresse:“ gehen auf STDERR (farbig hervorgehoben). Bei Fehlern können .obj, .mqo und .dmp trotzdem teilweise entstehen und sind dann nicht verwendbar.
Die <adresse> der Befehle im Objektfile beginnen immer bei 0000. Ein Operand von **start** ändert die Adressen nicht; der Wert steht nur in der Kennzeichenzeile 21 0000 <zahl> (siehe 3.7).

Der Assembler erzeugt bearbeitet Dateien im Verzeichnis der *.asm -Datei:
Projekt_1/bsp_5.16/

bsp_5_16.asm - Quelltext
bsp_5_16.dmp - formatierte Speicherbelegung (Listing: Adresse, Maschinenwort, Label, Mnemonic, Operand, Kommentar); entspricht dem Listing auf STDOUT
bsp_5_16.log - Abschnitte, soweit vorhanden: Literaltabelle, EQU-Definitionen, ABS-Definitionen, Symboltabelle (Symbol, RelAddr, Adresse, original Operation) und Adresstabelle (nach Adresse sortiert)
bsp_5_16.mqo - aufbereiteter Quelltext für c82load, der ihn mit dem Maschinencode kombiniert
- für c82simi, der ggf. bei der Einzelschrittverarbeitung die Kommentare einblendet
bsp_5_16.obj - mit c82load ladefähiger Objektcode K1K2
bsp_5_16.sym - Symboldatei , wird von c82dump benutzt, um zum Trommeldump Symbole zuzuordnen. Je Zeile <symbol>,<adresse> (vierstellig oktal, in Quelltextreihenfolge, einschließlich Literalsymbolen wie l0001)
bsp_5_16.test - Trommeldatei im Format des Interpreters (Adresse: Wert # Quelltextzeile) mit Kopf (Trommeldatei:, #DOK-0001, #SPEI-0001 0000-0000, #PROT STUFE=2) und Fuß (Tastenfolge:, H G R), die für den jeweiligen Fall ergänzt bzw. erweitert werden müssen. Zeilen gibt es nur für Wörter mit K1=00 oder 01 (Befehle, dw, da, dz, dt, dto, dtm, Nullwort); db, equ, start und abs erzeugen keine Zeile; Grundlage für Testaufträge, s. C8205-Interpreter-Durchführung_eines_Tests.md, Abschnitt 2a

Vor jedem Lauf löscht der Assembler die Dateien .log, .obj, .mqo, .sym, .test und .dmp des Projekts und erzeugt sie neu. Die früher genannten Endungen .mqt (modifizierter Quelltext) und .l82 (Objektcode für den C8205Simulator) erzeugt der aktuelle Assembler nicht mehr. Dasselbe gilt für .mac: Der Quellcode legt keine solche Datei an.

2. Sprachdefinition

<zahl> ::= <oktalzahl> | <dezimalzahl>
<oktalzahl> ::= oktalziffer==0 [oktalziffer [oktalziffer …]]
<oktalziffer> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7
<vorzeichen> ::= -
<dezimalziffer> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
<dezimalzahl> ::= [<vorzeichen>] dezimalziffer<>0 [dezimalziffer [dezimalziffer …]]
<alphazeichen> ::= abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ
<sonderzeichen> ::= % : + - / ` , . < " [> ' ~] # * (= ;) ? !

<wert> ::= <oktalzahl> | <dezimalzahl> | <symbol>
<rechenzeichen> ::= + | - | * | /
<speicherplatz> ::= Speicherstelle auf dem Trommelspeicher, fortlaufend nummeriert durch eine Oktalzahl mit dem Wertebereich 00000 bis 07777.
<symbol> ::= <adresssymbol>|<literalsymbol>| (leer)
<adresssymbol> ::= Zeichenfolge aus 1 bis 32 Zeichen:
<alphazeichen> [<alphazeichen> | <dezimalziffer> […]],
abgeschlossen durch Doppelpunkt
Sie wird vom Programmierer frei definiert. Der Unterstrich _ wird ebenfalls akzeptiert. Das erste Zeichen ist ein Buchstabe oder _, keine Ziffer. Erlaubt sind nur die ASCII-Buchstaben a–z, A–Z, die Ziffern 0–9 und _; Zeichen wie -, . oder ä sind unzulässig. Doppelt definierte Symbole, Symbole über 32 Zeichen und unzulässige Symbole sind Fehler.
<literalsymbol> ::= ‚l‘ (kleines L) gefolgt von einer Oktalziffernfolge von genau 4 Oktalziffern , abgeschlossen durch Doppelpunkt.
Sie wird vom Assembler automatisch aus dem Inhalt des Literalzählers vergeben.
<ausdruck> ::= <werteausdruck> | <literal>
<operand> ::= <werteausdruck> | <literal> | (leer)
<adressausdruck> ::= <wert><rechenzeichen>[<wert><rechenzeichen> …]
Einschränkungen: kein negatives Vorzeichen
Wertebereich nur von 00000 … 07777
Nur maximal ein Symbol erlaubt.
<werteausdruck> ::= <wert><rechenzeichen>[<wert><rechenzeichen> …]
Einschränkung: Bei dw und in #-Literalen sind keine Symbole zulässig
(auch keine abs-Symbole), dort nur Zahlen; Wertebereich -2^32 … 2^33-1.
Adressen werden mit da oder @-Literalen geschrieben.
<literal> ::= **#**<werteausdruck> | **@**<adressausdruck>
Literal bedeutet, dass der nummerische Wert von <werteausdruck> oder <adressausdruck> auf einem vom Assembler selbst festgelegten <speicherplatz> gespeichert wird. Diesem Speicherplatz wird vom Assembler ein <literalsymbol> zugeordnet.
<adresse> ::= nummerischer Wert eines <symbol>, der einen <speicherplatz> adressiert.
<kommentar> ::= **;** (Semikolon) gefolgt von ASCII-Zeichen. | (leer)
Innerhalb der \-Zeichen von **dt** wird ; nicht als Kommentar betrachtet.
<trenner> ::= Tabulator[<trenner>[...]] | Leertaste<trenner>[...]]
<befehl> ::= [<symbol>] <trenner><operationscode> [<trenner><operand>] [<trenner><kommentar>]

Alle Befehle müssen auf einer Zeile stehen; eine Zeile darf höchstens 255 Zeichen lang sein.

<operationscode> ::= <maschinenbefehl> | <assembleranweisung>
<assembleranweisung>::=start | end | db | dbn | dw | da | dt | dto | dtm | dz | equ | equ10 | equ40 | equ100 | abs
<maschinenbefehl> ::= <oktalziffer><oktalziffer><oktalziffer>

<quelltext> ::=<befehl>[<befehl> […]]

3. Assembleranweisungen

3.1 Definiere Bereich

[<symbol>] **db** <zahl>

Hält <zahl> Speicherplätze frei, ohne ihren Inhalt zu definieren

3.2 Definiere Bereich auf Null

[<symbol>] **dbn** <zahl>

Hält <zahl> Speicherplätze frei, setzt ihren Inhalt auf 0.

3.3 Definiere Wert

[<symbol>] **dw** <werteausdruck>

- Speichert den nummerischen Wert von <werteausdruck> auf dem <speicherplatz> Zulässig ist der Bereich -2^32 bis 2^33-1 (33-Bit-Wort), sonst meldet der Assembler „Operand ist ungültig“. Negative Werte werden im Zweierkomplement gespeichert (dw -5 ergibt 77777777773). Symbole sind im <werteausdruck> von dw nicht zulässig; Adressen schreibt man mit da.
- Es wird keine Hauptleitadresse addiert.

[<symbol>] **dw** <literal>

- Erzeugt am dem Programmende automatisch aus <literal> eine Definition der Art: <literalsymbol> DW <werteausdruck>.
- Speichert die <adresse> **von** <literalsymbol> auf dem aktuellen <speicherplatz>.
- Es wird keine Hauptleitadresse addiert.

3.4 Definiere Adresse

[<symbol>] **da** <adressausdruck>

Speichert den nummerischen Wert von <adressausdruck> auf dem <speicherplatz> verschoben um «9 in den Adreßstellen.
Zum <adressausdruck> wird die 1. Hauptleitadresse addiert.

[<symbol>] **da** <literal>

- Erzeugt am dem Programmende automatisch aus <literal> eine Definition der Art: <literalsymbol> da <adressausdruck>.
- Zum <adressausdruck> wird die 1. Hauptleitadresse addiert.
- Nimmt stattdessen die <adresse> von <literalsymbol> in den befehl, der das <literal> enthält.

3.5 Definiere Text zur Ausgabe von Zeichens

[<symbol>] **dt**[**o**|**m**] \<alphazeichen> | <leerzeichen> | <dezimalziffer> <sonderzeichen> [<alphazeichen> | <leerzeichen> | <dezimalziffer> | <sonderzeichen>[...]]\

- gespeichert wird ein Code gemäß R300 – Code Ausgabeadresse C8205 (siehe R300-code.pdf). Die Abspeicherung erfolgt als Folge von Speicherplatzinhalten in der Form (oktal): 0000 xxxx 030, wobei xxxx die Ausgabeadresse ist.
  Die Zeichen „\“ am Anfang und am Ende sind nur erforderlich, wenn der Text Leerzeichen enthält oder mit # beginnt. Ein führendes # kennzeichnet ein Literal, keinen Text; der Assembler meldet dafür einen Fehler (richtig: dt \#ab\, dto \#x\). Ein # innerhalb des Textes (dt a#b) ist zulässig.

dt ist für längere, zusammenhängende Textausgaben gedacht. Sein Vorteil: Der Text lässt sich ohne ausgeschriebene Umschaltzeichen definieren, der Quelltext bleibt gut lesbar. Für

verstreute Einzelzeichen ist **dt** nicht geeignet, dafür nimmt man **dz** (3.6) und schreibt alle Umschaltzeichen explizit.

Der Assembler setzt **dt** zur Assemblerzeit in Maschinenbefehle um und kennt den Zustand KB/GB zur Laufzeit nicht. Er führt den Zustand nur in der Reihenfolge des Quelltexts weiter. Beim allerersten **dt** des Quelltexts nimmt er KB an. Jedes weitere **dt** setzt den Zustand des vorangehenden **dt** fort und fügt nur dort ein Umschaltzeichen ein, wo sich der Zustand innerhalb des Textes ändert.

Daraus folgt: Vor jeder zusammengehörenden Folge von **dt**-Anweisungen muss ein Umschaltbefehl durchlaufen werden, der exakt auf den Zustand des ersten Zeichens dieser Folge schaltet (**dz $inkb**, **dz $ingb** oder **dtm** als erste Anweisung der Folge). Das gilt für jede Folge neu. Es kann dabei ein eigentlich unnötiger Umschaltbefehl entstehen. Der ist beabsichtigt, weil sich Assemblerzeit und Laufzeit nicht synchronisieren lassen. Umschaltzeichen mit **dz** zwischen zusammengehörenden **dt**-Anweisungen sind nicht zulässig, sie bringen den Zustand des Assemblers durcheinander.

Praktischer Hinweis dazu: **dtm** fügt immer vor dem ersten Zeichen ein Umschaltzeichen passend zum ersten Buchstaben ein. Damit ist es die natürliche erste Anweisung einer **dt**-Folge. Es ersetzt das von Hand gesetzte **dz $inkb** oder **dz $ingb** davor.

dto verhält sich wie **dt**, außer dass generell KEINE Umschaltzeichen *automatisch* eingefügt werden. Das bedeutet, in einer dto-Anweisung kann man KEINE gemischte Groß- und Kleinschreibung definieren ! (auch keine Zahlen und Großbuchstaben usw.)
dtm verhält sich wie **dt**, außer dass vor dem ersten Zeichen generell mit Umschaltzeichen, dem ersten Buchstaben entsprechend, automatisch eingefügt wird.

3.6 Definiere Zeichen (spezielle Sonderzeichen)

[<symbol>] **dz** $wrzl | $sm | $inkb | $ingb | $irrz | $irrs | $trans | $bm | $vorl | $irrb | $wr | $tab | $zr | % | : | + | ` | / | , | . | -

- gespeichert wird ein Code gemäß R300 – Code Ausgabeadresse C8205 (siehe Tabelle 2). Die Abspeicherung erfolgt auf einem Speicherplatz in der Form (oktal): 0000 ssss 030. Eine automatisch Ergänzung von Umschaltzeichen wie bei **dt** erfolgt hier generell NICHT.

Findet man beispielsweise **dz $wrzl** zu kompliziert, kann man das mit einem Makro vereinfachen:

```
WRZL:   MACRO
            dz      $wrzl  ; Zeilenschaltung
        MEND
```

(In der mitgelieferten c82ass.makros enthält WRZL zusätzlich die Anweisung ANWAHL 4700.)

Das Zeichen ; kann bei dz nicht angegeben werden, weil es einen Kommentar einleitet; der Operand für den Code 0150 ist : (vgl. Tabelle 2 und 3).

3.7 Beginn des Quellcodes

start [<zahl>]

- Nur als 1. Anweisung im <quelltext> zulässig, und nur einmal; ein Label vor start ist ein Fehler. Der Operand muss eine Konstante sein. Der Befehlszähler beginnt unabhängig vom Operanden bei 0.

Eine Maschinencodeausgabe erfolgt nicht; mit Operand wird im Objektcode die Kennzeichenzeile 21 0000 <zahl> ausgegeben.
Ist <zahl> angegeben, wird die 1. Leitadresse in den „Lochstreifen“ ausgegeben.
Es wird dann durch c82load auf diese 1.Leitadresse + Ladeadresse geladen.
Ladeadresse existiert dabei nur bei Laden mit c82load.

3.8 Definition Relativadresse

[<symbol>] **equ** <werteausdruck> | *

- Definiert für das <symbol> einen <werteausdruck>.
Der Befehlszähler wird auf diesen Wert gesetzt.
Ist ‚*‘ angegeben, wird dem Symbol dagegen der aktuelle Befehlszählerstand zugewiesen.
Eine Maschinencodeausgabe erfolgt nicht.
Im Objektcode für das „Kleine Eingabeprogramm“ wird der Kennzeichenbefehl für das Setzen der Relativadresse ausgegeben (KZ=20).
Das weitere Laden erfolgt auf 1.Leitadresse + Ladeadresse + Relativadresse.
Ladeadresse existiert dabei nur bei Laden mit c82load.
Die Anweisungen **equ100**, **equ40** und **equ10** runden den Befehlszähler (Operand *) oder den numerischen Operanden auf das nächste
Vielfache von 0100, 040 bzw. 010 (oktal) auf.

3.9 Definition Absolutadresse

[<symbol>] **abs** <oktalzahl> | <dezimalzahl>

- Definiert für das <symbol> eine absolute <adresse>.
Eine Objektcodeausgabe erfolgt nicht.
Eine Maschinencodeausgabe erfolgt nicht.
Beachte: Symbole, die durch **abs** definiert wurden, sind nicht verschieblich. Anweisungen, in denen ein solches Symbol im Adreßausdruck oder Werteausdruck vorkommt, werden mit K1K2=00 generiert !
abs kann mehrfach mit dem gleichen Wert definiert werden, ohne Fehler zu erzeugen, aber nicht mehrfach mit dem gleichen Symbol für unterschiedliche Werte.

3.10 Ende des verschieblichen Teils des Quelltextes

[<symbol>] **end**

- Hinter dieser Anweisung sollten nur noch **abs** oder **equ**-Anweisungen folgen.
Der Assembler ordnet hinter „**end**“ die automatisch erzeugten Literaldefinitionen ein, so dass sie noch zum verschieblichen Teil hinzukommen.
Ein nachfolgendes **equ** käme also HINTER die Literale.
Eine Maschinencodeausgabe erfolgt für „**end**“ nicht.

Tabelle 2: Sonderzeichen für **dz** – Anweisung

Spalte 1: String, der als Operand bei **dz** anzugeben ist
Spalte 2: 4 Oktalziffern, die die 8 Bit des 030 Ausgabecodes enthalten

$wrzl	02720
$sm	02720
$inkb	01170
$ingb	0570
$irrz	03770
$irrs	0160
$trans	01770
$bm	03520
$vorl	0
$irrb	0760
$zr	0320
$tab	01550
$wr	01120
%	01740
:	0150
+	0400
`	01600
/	01410
,	01530
.	0730
-	01000

Tabelle 3: Ausgabe-Umschlüsselung außer Sonderzeichen dz

Spalte 1: ascii -Kleinbuchstaben
Spalte 2: ascii -Großbuchstaben
Spalte 3: 4 Oktalziffern, die die 8 Bit des 030 Ausgabecodes enthalten
Spalte 4: ascii -code der Großbuchstaben
Spalte 5: ascii -code der Kleinbuchstaben

1	<	0010	60	49
2	"	0020	34	50
3	≈	0230	0	51
4	[	0040	91	52
5	>	0250	62	53
6	'	0260	39	54
7	≋	0070	0	55
8	~	0100	126	56
9	]	0310	93	57
0	#	0200	35	48
%	*	1740	42	37
Tabulator	Tabulator	1550	9	9
q	Q	1300	81	113
w	W	1660	87	119
e	E	0450	69	101
r	R	1110	82	114
t	T	1630	84	116
z	Z	1710	90	122
u	U	1440	85	117
I	I	0510	73	105
o	O	1060	79	111
p	P	1270	80	112
:	(	0150	40	58
+	=	0400	61	43
a	A	0610	65	97
s	S	1420	83	115
d	D	0640	68	100
f	F	0460	70	102
g	G	0670	71	103
h	H	0700	72	104
j	J	1210	74	106
k	K	1220	75	107
l	L	1030	76	108
`	;	1600	59	96
/	)	1410	41	47
SatzMarke	WRZL	2720	10	10
y	Y	1500	89	121
x	X	1470	88	120
c	C	0430	67	99
v	V	1650	86	118
b	B	0620	66	98
n	N	1050	78	110
m	M	1240	77	109
,	?	1530	63	44
.	!	0730	33	46
-		1000		45
Leertaste	ZwR	0320	32	32

4. Der Makro-Präprozessor des Assemblers

Der Präprozessor ist sehr einfach gehalten und hat eine starre Syntax. Er prüft, ob eine Operation ein Makroname ist. Die Prüfung erfolgt in der Textdatei c82ass.makros, die erweitert werden darf. Der Assembler sucht sie unter diesem festen Namen zuerst im Arbeitsverzeichnis des Aufrufs, sonst unter Sourcecode/c82ass.makros; fehlt sie an beiden Orten, bricht er mit einer Fehlermeldung ab. Zusätzlich können Makros direkt im Quelltext definiert werden (siehe Abschnitt 4.1).

Jeder der dort definierten Makros beginn mit dem Makronamen unmittelbar gefolgt von einem Doppelpunkt und danach (getrennt durch Tab oder Leerzeichen) das Schlüsselwort MACRO.
Müssen dem Makro Parameter übergeben werden, folgt nach MACRO, getrennt durch Leerzeichen, die Anzahl der Parameter. Es sind Maximal 9 Parameter zulässig.
Danach kommen die Definitionsanweisungen des Makros. Sie kann die formalen Parameter **$**x mit x=1..9 enthalten, die durch die Übergabeparameter ggf. mehrfach ersetzt werden.
Jeder Makro wird durch das Schlüsselwort MEND abgeschlossen (Leerraum davor ist beliebig).

Im Definitionsteil kann für jeden Parameter eine Prüfliste angegeben werden. Ist sie angegeben, darf der betreffende Parameter nur die Werte der Liste annehmen. Eine Prüfliste wird durch $Prx mit x=1..9 definiert. Sie muß auf einer Zeile für sich stehen.
Statt einer Werteliste kann hinter $PRx eine Typangabe stehen (z. B. $PR1: $TYPADR). Dann prüft der Assembler beim Makroaufruf, ob der übergebene Wert zum Typ passt; sonst meldet er einen Fehler „Parameter $x: Wert …“.
$TYPADR: eine symbolische Adresse (ein Label, z. B. marke) oder eine Sternadresse (*, *+3); keine Zahl.
$TYPKON: eine Oktalkonstante: nur Ziffern 0–7, beginnt mit 0 (z. B. 0100); 100 oder 09 sind Fehler.
$TYPABS: ein Symbol, das im Quelltext mit abs definiert ist (z. B. absx: abs 05).
Die Prüfung erfolgt vor der eigentlichen Übersetzung und ist deshalb nur syntaktisch: Bei $TYPADR wird nicht geprüft, ob das Label im Programm vorkommt (das meldet die spätere Symbolauflösung).

Die Definition der zu generierenden Anweisungen kann durch eine $IF-$ELIF-$ENDIF Struktur gesteuert werden. Sie kann mehrfach auftreten, darf aber nicht geschachtelt werden. Die Schlüsselworte $IF $ELIF $ENDIF müssen jeweils auf einer Zeile für sich stehen.
Hinter $IF und $ELIF muß mindestens einer, können bis zu 4 Bedingungsblöcke angegeben werden. Ein Bedingungsblock besteht aus 3 Strings: <wert1> <vergleichszeichen> <wert2>.
Als <vergleichszeichen> sind: >, <, =, >=, <= zugelassen. Jeder folgende Bedingungsblöcke ist durch **|** (für logisch ODER) oder **&** (für logisch UND) zu verknüpfen.
Achtung: Die Bedingungsblöcke haben eine IMPLIZITE Klammerung der Art:
(((<bedingungsblock1> [|&] <bedingungsblock2> **)** [|&] <bedingungsblock3> **)** [|&] <bedingungsblock4> **)**

Innerhalb eines Definitionsblockes sind Kommentare zugelassen. **#** leitet einen Kommentar ein, der nur in der Makrodefinition gilt. **#** muß auf einer Zeile für sich stehen.
; leitet einen Kommentar ein, der auch generiert wird. Es kann auf einer Ziele für sich oder in der Anweisung vorkommen.

Beispiel:

```
ANWAHL:  MACRO 1
 $PR1 4000 4100 4200 4300 4400 4500 4600 4700 K0 K1 K2 K3 LSL1 LSL2 LKL1 LKS1 LSS1 SW1I SW1O SW2O KONI
 # Generieren einer Anwahl für Zeichenkanäle
    $IF  $1  =  4000 | $1 = K7
      030 04000  ; 4000 oder K7
    $ELIF  $1  =  4100 | $1 = K6
      030 04100  ; 4100 oder K6
    $ELIF  $1  =  4200 | $1 = K5
      030 04200  ; 4200 oder K5
    $ELIF  $1  =  4300 | $1 = K4 | $1 = LKS1
      030 04300  ; 4300 oder K4 oder LKS1
    $ELIF  $1  =  4400 | $1 = K3 | $1 = LKL1 | $1 = SW2O
      030 04400  ; 4400 oder K3 oder LKL1 oder SW2 Output
    $ELIF  $1  =  4500 | $1 = K2 | $1 = LSL2 | $1 = SW1O
      030 04500   ; 4500 oder K2 oder LSL2 oder SW1 Output
    $ELIF  $1  =  4600 | $1 = K1 | $1 = LSL1 | $1 = LSS1
      030 04600  ; 4600 oder K1 oder LSL1 oder LSS1
    $ELIF  $1  =  4700 | $1 = K0 | $1 = SW1I | $1 = KONI
      030 04700  ; 4700 oder K0 oder SW1 Input oder KON Input
    $ENDIF
      MEND
```

Aufruf: **ANWAHL LSS1** erzeugt: 030 04600 ; 4600 oder K1 oder LSL1 oder LSS1
Der Makroaufruf im Assemblerprogramm darf ein Label haben:

```
input_loop:    ANWAHL   LSL1
               …
```

In einem solchen Fall wird generiert:

```
input_loop:    030     04600 ; 4600 oder K1 oder LSL1 oder LSS1
```

Der Makroaufruf selbst wird nicht als Kommentarzeile („; ANWAHL LSL1“) ausgegeben; er erscheint weder im Listing noch in .mqo, .dmp oder .test.

Enthält der Makro z.B. selbst eine Schleife mit einem Label:

```
BSP1:          MACRO
# Makro enthält eine Schleife
ML0001%:             211     …
               …
               000     ML0001%
               MEND
```

So wird bei einem Makroaufruf:

```
loop:          BSP1
```

folgende Anweisungsfolge generiert, wenn die erste zu generierende anweisung ein eigenes Label hat:

```
loop           equ     *
ML0001DKR 211     …
               …
               000     ML0001DKR
```

Hat die erste Anweisung kein eigenes Label, entfällt die Equ-Anweisung und die erste Anweisung erhält das Label des Makroaufrufes. Ausnahme: Ist die erste Anweisung equ, abs, start oder end, bleibt die Equ-Anweisung bestehen.

Ein Macro SOLLTE kein „normales“ Label der Assemblersprache enthalten, sonst kann er nur einmalig aufgerufen werden. Stattdessen muss ein generisches Label verwendet werden: genau ML, vier Ziffern und %, danach der Doppelpunkt (z. B. ML0004%:). Jedes andere Label mit % meldet den Fehler „Macrolabel muß mit ML beginnen …“. Bei jedem Aufruf wird das %-Zeichen durch drei Großbuchstaben ersetzt (z. B. ML0001DKR), sodass jeder Aufruf ein eigenes Label erhält.

Makro Aufrufparameter, die exakt 4 Ziffern enthalten, und nur diese (!!) , werden ggf. als 4-Ziffern-String geprüft (durch $PRx), sie erhalten bei der Benutzung (Ersetzung des formalen Parameters) automatisch eine Vornull.

ANWAHL 4600

erzeugt:

```
030     04600 ; 4600 oder K1 oder LSL1 oder LSS1
```

4.1 Makros im Quelltext

Makros können nicht nur in der Makrodatei **c82ass.makros**, sondern auch direkt im Quelltext (.asm) definiert werden. Die Definition hat dieselbe Syntax wie in der Makrodatei: Sie beginnt mit dem Makronamen, unmittelbar gefolgt von einem Doppelpunkt, danach folgt das Schlüsselwort MACRO (bei Bedarf mit der Anzahl der Parameter). Abgeschlossen wird sie durch MEND (zulässig sind auch MACROENDE und macroende). Der Doppelpunkt hinter dem Makronamen ist im Quelltext Pflicht.

Gültigkeit: Eine Definition darf an beliebiger Stelle im Quelltext stehen, auch vor start. Das Makro gilt im gesamten Quelltext, auch für Aufrufe, die vor der Definition stehen. Die Definition selbst erzeugt keinen Code.

Definitionsteil: Es gelten alle Regeln dieses Abschnitts (formale Parameter $x, Prüflisten $PRx, $IF-$ELIF-$ENDIF, Kommentare mit # und ;, generische Labels mit %). Ein Makro darf andere Makros aufrufen, gleichgültig ob sie in der Makrodatei oder im Quelltext definiert sind, aber nicht sich selbst.

Gleicher Name wie in der Makrodatei: Das Makro im Quelltext ersetzt das gleichnamige Makro der Makrodatei, jedoch nur für diesen Assemblerlauf; die Makrodatei selbst bleibt unverändert. Der Assembler gibt dazu eine Warnung aus. Die Ersetzung wirkt auch dort, wo ein anderes Makro das ersetzte Makro aufruft.

Fehler: Ein Makroname darf im Quelltext nur einmal definiert werden. Weitere Fehler sind eine MACRO-Zeile ohne Namen, eine Definition innerhalb einer anderen Definition (Schachtelung ist nicht erlaubt), ein fehlendes MEND und ein MEND ohne zugehöriges MACRO. Der Assembler meldet den Fehler und bricht nach Pass 0 ab.

Beispiel:

```
ZEILE2:  MACRO
         dz  $wrzl  ; Zeilenschaltung
         dz  $wrzl  ; Zeilenschaltung
         MEND
```

Der Aufruf ZEILE2 erzeugt die beiden Anweisungen dz $wrzl.

Soll das Makro WRZL der Makrodatei nur für ein Programm anders arbeiten, wird es im Quelltext neu definiert:

```
WRZL:    MACRO
         dz  $wrzl  ; Zeilenschaltung
         dz  $wrzl  ; Zeilenschaltung
         MEND
```

Der Assembler meldet dazu: Warnung: Makro WRZL aus der Makrodatei wird für diesen Lauf durch die Definition im Quelltext ersetzt.

5. Compilierungsbeispiel

c82ass.sh -p Projekt_1/bsp_5.16 -s bsp_5_16

5.1. Quelltext bsp_5_16.asm und Assemblerlauf

```
Projektverzeichnis: Projekt_1/bsp_5.16
Benutze für -s die Datei: bsp_5_16
Startadresse: 0000
C8205 Assembler C82aas Version 0.2

Pass 0
Verarbeite 'c82ass.makros', Erzeuge Macroersetzungen

Pass 1
Löse Literale auf

Pass 2
Löse Symbole auf nach Projekt_1/bsp_5.16/bsp_5_16.log und Projekt_1/bsp_5.16/bsp_5_16.sym

Pass 3
Erzeuge Kennzeichen-Befehle in Objektdatei Projekt_1/bsp_5.16/bsp_5_16.obj
; C8205 Heft 2 Programmierung in Maschinensprache
; Beispiel: 5.16 Umspeichern eines Vektors
;
                        start
                        000     beginn
aqv:                    da      qvektor    ; Adresse der ersten Komponente Quellvektor
dqv:                    dw      05000    ; Dimension von qvektor gezählt in Adreßeinheiten
azv:                    da      zvektor    ; Adresse der ersten Komponente Zielvektor
adreinheit:             dw      01000    ; Adresseinheit
spbef:                  dw      0240    ; Speicherbefehl
i:                      db      1    ; Arbeitszelle für i
beginn:                 241     i    ; I:=0
schleife:               210     azv    ; ac:=(azv+i)
                        210     spbef    ; ac:=(azv+i)240
                        240     genbef    ; Arbeitszelle genbef:= (qzv+i) 240
                        211     i    ; ac:=<i>
                        210     aqv    ; ac:=qvektor+i
;  251: Laden des Inhaltes einer Speicherstelle, deren Adresse im AC steht in den AC.
                        251     0    ; ac:=<qvektor+i>
genbef:                 db      1    ; hier steht der Befehl 240 (qzv+i)  Speichern
                        211     i    ; ac:=<i>
                        210     adreinheit    ; ac:=i+1
                        240     i    ; i:=ac
                        330     dqv    ; vergleicher 3, falls i == <dqv>
                        170     halt    ; sprung bei gleich
                        000     schleife
halt:                   020     0    ;halt
qvektor:                dw      1
                        dw      2
                        dw      3
                        dw      4
                        dw      5
                        equ     0100
zvektor:                db      5
ende:                   end


Nach dem Listing meldet der Assembler:

Ausgabe als Minimalprogramm-Objektdatei nach: Projekt_1/bsp_5.16/bsp_5_16.obj
Ausgabe der formatierten Speicherbelegung nach: Projekt_1/bsp_5.16/bsp_5_16.dmp
Ausgabe der Testdatei (Spalten 1,3,4,5 fuer K1=00/01) nach: Projekt_1/bsp_5.16/bsp_5_16.test
```

5.2. Durch c82ass erzeugter Objektcode (K1K2-Programmcode) für „Kleines Eingabeprogramm“

```
0000: 01 0007 000.
0001: 01 26000.
0002: 00 00000005000.
0003: 01 100000.
0004: 00 00000001000.
0005: 00 00000000240.
0006: 20 0007.
0007: 01 0006 241.
0010: 01 0003 210.
0011: 01 0005 210.
0012: 01 0016 240.
0013: 01 0006 211.
0014: 01 0001 210.
0015: 00 0000 251.
0016: 20 0017.
0017: 01 0006 211.
0020: 01 0004 210.
0021: 01 0006 240.
0022: 01 0002 330.
0023: 01 0025 170.
0024: 01 0010 000.
0025: 01 0000 020.
0026: 00 00000000001.
0027: 00 00000000002.
0030: 00 00000000003.
0031: 00 00000000004.
0032: 00 00000000005.
      20 0000 0100.
0100: 20 0105.
/
```